

Security Audit

OpenZK 5th (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	31
Conclusion	32
Our Methodology	33
Disclaimers	35
About Hashlock	36

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The OpenZK team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

OpenZK is an innovative Layer 2 (L2) network project that leverages Zero-Knowledge (ZK) Rollup technology to enhance Ethereum's scalability and transaction efficiency while maintaining high security. By integrating native ETH staking and restaking capabilities, OpenZK not only bolsters scalability but also creates new revenue streams for users. The platform's leadership includes co-founder Dave Sandor, formerly an Executive Director at Goldman Sachs Asia-Pacific, co-founder and CTO Lucas Cullen, an early contributor to Ethereum's development, and also co-founder Jenna Wayne. OpenZK distinguishes itself with a Dual Gas Fee Mechanism, allowing users to pay gas fees with its native and protocol tokens, enhancing network flexibility and creating sustained demand for its protocol token.

Project Name: OpenZK

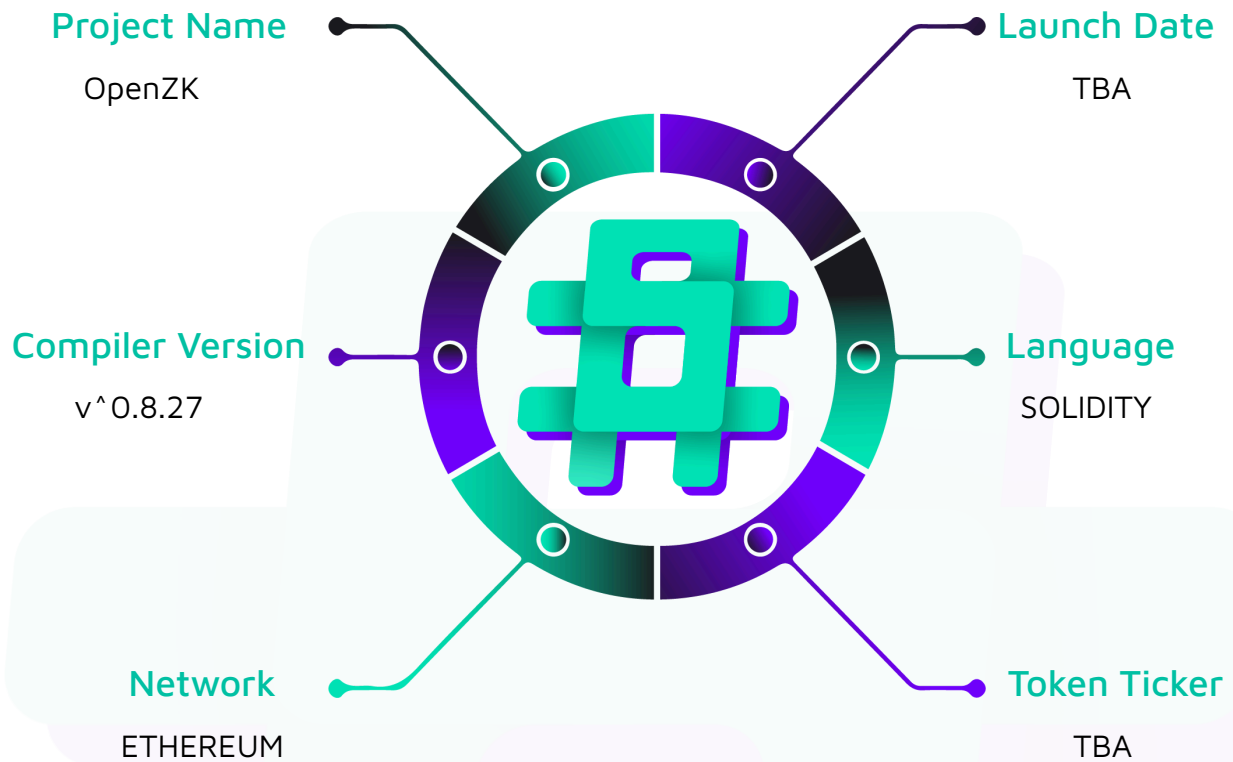
Project Type: DeFi

Compiler Version: ^0.8.27

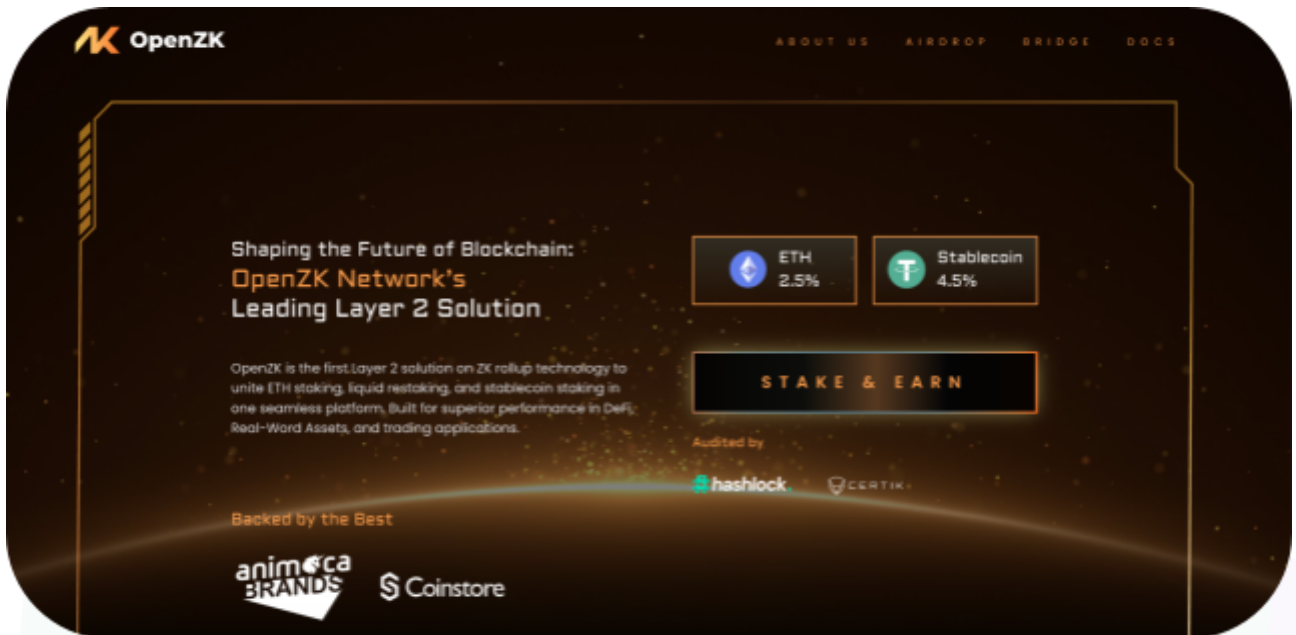
Website: <https://www.openzk.net/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the OpenZK project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	OpenZK Smart Contracts
Platform	Ethereum / Solidity
Audit Date	May, 2025
Contract 1	BridgeTokenMiddleware.sol
Contract 2	Staking.sol
Contract 3	YieldFarm.sol
Contract 4	wozETH.sol
Contract 5	ApprovalPaymaster.sol
Contract 6	GeneralPaymaster.sol
Audited GitHub Commit Hash 1	5e4e30c56b018c87760141a5db77e16341f3c309
Audited GitHub Commit Hash 2	a11fbf6dc98bab10020b0b1f487446aa68d164aa
Audited GitHub Commit Hash 3	fa79baa21a6fa83ec5ce80e25f197d3257c2860a
Fix Review GitHub Commit Hash 1	7d9a81effd39489f3b63a4e4b4bd0cd5aa2c10a1
Fix Review GitHub Commit Hash 2	c119d879eb45d049395c8b75438ef039fa55be75
Fix Review GitHub Commit Hash 3	6e2f39e078615e97339e5651cc36643bc158e059

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

3 Medium severity vulnerabilities

6 Low severity vulnerabilities

4 Gas Optimisations

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BridgeTokenMiddleware.sol - Allows users to: - Bridge ERC-20 tokens from L1 to L2 - Pay gas fees in ETH for L2 transactions - Receive refunds for any excess ETH - Allows operators to: - Add or remove supported ERC-20 tokens - Allows admins to: - Sweep tokens or ETH from contract	Contract achieves this functionality.
Staking.sol - Allows users to: - Deposit tokens into the staking pool - Check in to participate in a reward epoch - Queue withdrawals of their staked tokens after a lock period - Process previously queued withdrawals once the cliff time has passed	Contract achieves this functionality.
YieldFarm.sol - Allows users to: - Harvest rewards earned per epoch based on their stake in the Staking contract.	Contract achieves this functionality.

wozETH.sol - Allows users to: - Deposit ETH and receive equivalent wozETH tokens - Redeem wozETH tokens to withdraw ETH.	Contract achieves this functionality.
--	--

Code Quality

This audit scope involves the smart contracts of the OpenZK project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the OpenZK project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] BridgeTokenMiddleware#bridgeToken() - Can Cause Loss for Users

Description

The BridgeTokenMiddleware contract enforces a fixed 5.27% ETH slippage buffer in `_convertToEthAmount()`, even though the actual swap in RocketPoolVaultBase only requires 0.3% slippage. This forces users to overpay by ~5% ETH unnecessarily, leading to avoidable financial loss.

Vulnerability Details

The BridgeTokenMiddleware contract contains a slippage-related issue in the `_convertToEthAmount()` function, which is called during the `bridgeToken()` operation. When bridging tokens, the contract calculates the required ETH amount by applying a fixed 5.27% slippage buffer ($1_000_000 / 950_000$). However, this slippage is significantly higher than the actual slippage tolerance (0.3%) used in the subsequent `_stakeNative()` operation, which interacts with the LiquidityManager and RocketPoolVaultBase for swapping ETH to rETH.

The issue arises because the `_convertToEthAmount()` function unnecessarily inflates the ETH requirement by ~5.27%, even though the actual swap in RocketPoolVaultBase only requires a 0.3% slippage tolerance. This means that even if the actual slippage during the swap is negligible (e.g., 0.1% or lower), users are still forced to pay an additional ~5% ETH upfront, leading to an inefficient overpayment. The excess ETH is not refunded, resulting in unnecessary financial loss for users.

```
function _convertToEthAmount(uint256 _baseTokenAmount) internal returns (uint256) {  
  
    uint256 nativeCoinAmount = IERC20(native).totalSupply();  
    uint256 lmValue = liquidityManager.virtualBalance();  
  
    if (lmValue == 0) {
```

```

        return _baseTokenAmount;

    }

    return _baseTokenAmount * lmValue * 1_000_000 / (nativeCoinAmount * 950_000); //
cover for swap slippage

}

```

```

function _swapToUnderlying(uint256 amount) internal returns (uint256) {

    // Get the current balance of this contract in ETH

    uint256 balanceBefore = address(this).balance;

    uint256 assetBalanceBefore = super._totalAssets();

    uint256 _idealTokensOut = amount;

    uint256 _minTokenOut = amount * 1e15 * 997 / _oracle.getValueInEth(_underlying);
// 0.3% slippage

    // Swap ETH to rETH

    IRocketPoolRouter(_router).swapTo{value: amount}({

        uniswapPortion,

        balancerPortion,

        _minTokenOut,

        _idealTokensOut

    });
}

```

Impact

Users bridging tokens via `bridgeToken()` will lose ETH unnecessarily due to the inflated slippage calculation.

Recommendation

The slippage calculation in `_convertToEthAmount()` should be adjusted to match the actual slippage tolerance used in the swap operation (0.3%). Instead of using a hardcoded 5.27% buffer, the function should apply a 0.3% slippage factor

Status

Resolved

[H-02] BridgeTokenMiddleware#bridgeToken() - Account Abstraction Wallet Users Can Lose Funds During Bridging

Description

The BridgeTokenMiddleware contract's bridging function assumes `msg.sender` will have the same address on L2, which fails for Account Abstraction (AA) wallets that use different addresses across chains. This causes bridged tokens to be sent to an unreachable address on L2, permanently locking user funds.

Vulnerability Details

The BridgeTokenMiddleware contract's bridging functionality contains a critical flaw when interacting with Account Abstraction (AA) wallets. The vulnerability arises in the `_bridgeErc20()` function, where the `_l2Receiver` parameter is set to `msg.sender` without accounting for potential address differences across chains for AA wallet users. Since AA wallets can have different addresses on different chains, this implementation may cause bridged tokens to be sent to an address the user does not control on the destination chain, resulting in permanent loss of funds.

```
function bridgeToken(address token, uint256 amount, uint256 _l2GasLimit) external payable
nonReentrant {

    require(isAllowedToken[token], "BridgeWrap: token not supported");
    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    // mint gas for layer2 payment

    uint256 gasCost = l2TransactionBaseCost(tx.gasprice, _l2GasLimit, 800);
    uint256 neededEth = _convertToEthAmount(gasCost);
```



```

    require (msg.value >= neededEth, "BridgeWrap: not enough ETH to cover the gas
cost");

    uint256 gasMinted = _stakeNative(neededEth);

    // bridge to L2

    _bridgeErc20(msg.sender, token, amount, _l2GasLimit, 800, gasMinted);

    emit BridgeToken(token, amount, _l2GasLimit, 800, gasMinted);

    // refund the remaining ETH to the user

    (bool success, ) = payable(msg.sender).call{value: msg.value - neededEth}("");

    require(success, "BridgeWrap: refund failed");

}

```

Impact

AA wallet users risk losing their tokens permanently when bridging to Layer 2 (L2). The tokens will be locked on the destination chain under an address derived from a different key scheme than the user's AA wallet, making recovery impossible. This affects all users employing smart contract wallets (e.g., ERC-4337 wallets) or multi-chain AA solutions where address derivation differs between L1 and L2. Given the irreversible nature of cross-chain transactions, this constitutes a high-severity issue that could lead to significant financial losses for affected users.

Recommendation

To prevent fund loss for AA wallet users, the contract should implement an explicit address resolution mechanism. Instead of hardcoding msg.sender as the recipient, users should provide their intended L2 address as a parameter, with appropriate validation to ensure they retain control. Additionally, the contract could integrate with AA standards like ERC-4337 to verify address ownership across chains.

Status

Resolved

[H-03] wozETH#withdraw() - Faulty ETH Transfer Implementation Can Permanently Lock User Funds

Description

The wozETH contract's withdrawal mechanism contains a critical flaw in its fund transfer logic that can lead to permanent loss of user funds. The current implementation burns user tokens before attempting to transfer ETH, and fails to properly handle transfer failures, creating scenarios where users can lose access to both their wrapped tokens and the underlying ETH.

Vulnerability Details

The `withdraw()` function contains a critical vulnerability in its ETH transfer implementation due to its use of an unchecked low-level call. By performing the token burn before attempting the ETH transfer and failing to verify the success of the call operation, the function creates a scenario where failed transfers can result in permanent loss of user funds. When the ETH transfer fails - which can occur for various common reasons including recipient contract reverts, gas limitations, or network issues - the transaction completes without reverting while the user's tokens have already been burned and the corresponding ETH remains stuck in the contract. This implementation flaw leaves users with no recourse to recover either their burned tokens or the trapped ETH, as the contract provides no mechanism to handle such failures or restore the system state.

```
function withdraw(uint256 amount) external {  
    require(balanceOf(msg.sender) >= amount, "wozETH: INSUFFICIENT_BALANCE");  
    _burn(msg.sender, amount);  
  
    msg.sender.call{ value: amount }("");  
}
```

Impact

When users attempt to withdraw their ETH by burning wozETH tokens, the flawed implementation creates a situation where funds can be permanently lost without any malicious action required. The silent failure of ETH transfers means users unknowingly



sacrifice their tokens while receiving nothing in return, with the corresponding ETH becoming permanently trapped in the contract.

Recommendation

The proper remediation for this vulnerability involves validating the return value of the low-level call operation when transferring ETH. The contract must explicitly check the boolean success value returned by the call() function and revert the transaction if the transfer fails.

Status

Resolved

Medium

[M-01] ApprovalPaymaster#postTransaction() - Paymaster can block user refunds by omitting postTransaction logic

Description

The ApprovalPaymaster contract fails to implement refund logic in the `postTransaction` function, causing users to lose excess tokens used for gas fees. This breaks zkSync's paymaster expectations and can lead to unintended financial loss.

Vulnerability Details

The ApprovalPaymaster smart contract implements the IPaymaster interface from the zkSync system contracts, enabling ERC-20-based gas sponsorship through the approval-based flow. However, the `postTransaction` function, which is a mandatory part of the paymaster interface, is currently implemented as a no-op and fails to fulfill its intended role of finalizing post-execution logic, including the distribution of refunds.

According to the zkSync protocol, `postTransaction` is invoked after transaction execution and is expected to handle important finalization tasks such as refunding unused gas fees, cleaning up state, and emitting audit-relevant events. In this contract, the function is declared but entirely empty, making it impossible for any refund logic to execute—even if gas was overpaid by the user or other token-based incentives are meant to be returned. Additionally, the `context` parameter, returned from `validateAndPayForPaymasterTransaction`, is not used or validated, which precludes passing relevant information between validation and post-execution phases.

```
function postTransaction(  
    bytes calldata _context,  
    Transaction calldata _transaction,  
    bytes32,  
    bytes32,  
    ExecutionResult _txResult,  
    uint256 _maxRefundedGas  
  
    ) external payable override onlyBootloader {}
```

Impact

The absence of refund logic in the `postTransaction` function can lead to users being overcharged for transactions.

Recommendation

The `postTransaction` function should be implemented to support refund logic in accordance with the zkSync paymaster interface specification.

Status

Acknowledged

[M-02] ApprovalPaymaster#validateAndPayForPaymasterTransaction - Use `safeTransferFrom()` instead of `transferFrom()`

Description

The contract uses `IERC20.transferFrom` without verifying its return value, which can silently fail for non-standard tokens that return false instead of reverting.

Vulnerability Details

Tokens that do not comply with the ERC20 specification could return false from the transfer function call to indicate the transfer fails, while the calling contract would not notice the failure if the return value is not checked. Checking the return value is a requirement, as written in the EIP-20 specification: "Callers MUST handle false from returns (bool success). Callers MUST NOT assume that false is never returned!"

Some tokens do not return a bool on ERC20 methods. This will make the call break, making it impossible to use these tokens.

```
function validateAndPayForPaymasterTransaction(  
    bytes32,  
    bytes32,  
    Transaction calldata _transaction  
)  
    external  
    payable
```

```

onlyBootloader
returns (bytes4 magic, bytes memory context)

{
    // By default we consider the transaction as accepted.
    magic = PAYMASTER_VALIDATION_SUCCESS_MAGIC;

    // Note, that while the minimal amount of ETH needed is tx.gasPrice * tx.gasLimit,
    // neither paymaster nor account are allowed to access this context variable.

    uint256 requiredETH = _transaction.gasLimit *
        _transaction.maxFeePerGas;

    try

        IERC20(token).transferFrom(userAddress, thisAddress, amount)

    {} catch (bytes memory revertReason) {

        // If the revert reason is empty or represented by just a function selector,
        // we replace the error with a more user-friendly message
        if (revertReason.length <= 4) {

            revert("Failed to transferFrom from users' account");

        } else {

            assembly {

                revert(add(0x20, revertReason), mload(revertReason))

            }

        }

    }

}

```

Impact

It would not revert even though the transaction failed.

Recommendation

Use SafeTransferLib or `SafeERC20`, replace `transferFrom()` with `safeTransferFrom()` when transferring `ERC20` tokens.

Status

Resolved

[M-03] BridgeTokenMiddleware#bridgeToken() - Fee-on-transfer tokens can cause inflated L2 minting for users

Description

The contract fails to handle fee-on-transfer (FOT) tokens properly, assuming full transfer amounts without verifying the actual tokens received. This can lead to incorrect bridging amounts.

Vulnerability Details

The BridgeTokenMiddleware contract uses IERC20.safeTransferFrom in various locations, including within the bridgeToken and _bridgeErc20 functions, to handle ERC20 transfers and approvals. While safeTransferFrom is generally secure, the contract assumes that the transferred or approved amount exactly reflects the intended value (_amount). This assumption breaks down for fee-on-transfer (FOT) tokens, which deduct a percentage fee upon transfer or when invoking transferFrom.

In the bridgeToken function, the user-supplied token amount is transferred to the middleware contract via IERC20(token).safeTransferFrom(msg.sender, address(this), amount). However, if the token charges a fee on transfer, the actual amount received by the contract will be less than the requested amount. Despite this discrepancy, the contract proceeds to bridge the full amount via _bridgeErc20, which sets approvals and generates the deposit calldata under the assumption that the full token amount is available.

```
function bridgeToken(address token, uint256 amount, uint256 _l2GasLimit) external payable
nonReentrant {
    require(isAllowedToken[token], "BridgeWrap: token not supported");

    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);

    // mint gas for layer2 payment

    uint256 gasCost = l2TransactionBaseCost(tx.gasprice, _l2GasLimit, 800);

    uint256 neededEth = _convertToEthAmount(gasCost);
```



```

    require (msg.value >= neededEth, "BridgeWrap: not enough ETH to cover the gas
cost");

    uint256 gasMinted = _stakeNative(neededEth);

    // bridge to L2

    _bridgeErc20(msg.sender, token, amount, _l2GasLimit, 800, gasMinted);

    emit BridgeToken(token, amount, _l2GasLimit, 800, gasMinted);

    // refund the remaining ETH to the user

    (bool success, ) = payable(msg.sender).call{value: msg.value - neededEth}("");

    require(success, "BridgeWrap: refund failed");

}

```

Impact

In the presence of fee-on-transfer (FOT) tokens, the contract assumes that the full amount specified by the user is received by the contract. However, due to transfer fees, the actual amount received is lower. Since the `_bridgeErc20` function uses the originally requested amount for minting tokens on L2, the user ends up receiving more tokens on the destination chain than they effectively deposited on L1.

Recommendation

To ensure compatibility with fee-on-transfer tokens, the contract should not rely on the input amount parameter to reflect the actual number of tokens received. Instead, after executing `safeTransferFrom`, it should calculate the net tokens received by comparing the token balance before and after the transfer.

Status

Resolved

Low

[L-01] BridgeTokenMiddleware#recoverEth - Deprecated Ether transfer function

Description

The recoverEth function uses the .transfer function to transfer native coins. The Istanbul update made some changes to the EVM, which made the .transfer function deprecated for the ETH transfer.

Recommendation

Use the .call function for the ETH transfer and check the return value.

Status

Resolved

[L-02] ApprovalPayMaster, GeneralPaymaster - Floating pragma

Description

The contract uses pragma solidity ^0.8.0 This can allow the contracts to be deployed with the wrong pragma version which is different from the one the contracts were tested with.

Recommendation

Lock the pragma version.

Status

Acknowledged

[L-03] ApprovalPayMaster, GeneralPaymaster- Use Ownable2Step version rather than Ownable version

Description

Ownable2StepUpgradeable and Ownable2Step prevent the contract ownership from



mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using `Ownable2StepUpgradeable` and `Ownable2Step` from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Acknowledged

[L-04] Staking- Missing zero address/value checks

Description

The `setWithdrawDelay()`, `epoch1Start`, `epochDuration`, `stakingToken` are missing zero address/value checks for the input parameters values.

```
constructor(uint256 _epoch1Start, uint256 _epochDuration, address _ozkTokenAddress,
address _adminAddr, address _moderatorAddr) {

    epoch1Start = _epoch1Start;

    epochDuration = _epochDuration;

    stakingToken = _ozkTokenAddress;

    _grantRole(DEFAULT_ADMIN_ROLE, _adminAddr);

    _grantRole(MODERATOR_ROLE, _moderatorAddr);

}

function setWithdrawDelay(uint256 newDelay) public onlyRole(MODERATOR_ROLE) {

    withdrawDelay = newDelay;

    emit WithdrawDelaySet(newDelay);
```

```
}
```

Recommendation

Add checks if the input parameters values are zero.

Status

Resolved

[L-05] Contracts - Unused Dependencies and Redundant Reentrancy Protection

Description

The BridgeTokenMiddleware contract imports AggregatorV3Interface from Chainlink but does not use it, increasing deployment costs without providing any functionality. Additionally, the wozETH contract inherits from ReentrancyGuard and doesn't use it anywhere.

Recommendation

Delete the AggregatorV3Interface import since it serves no purpose and remove ReentrancyGuard if it is of no use.

Status

Resolved

[L-06] BridgeTokenMiddleware:l2TransactionEthCost - Dead code

Description

It is recommended to keep the production repository clean to prevent confusion and the introduction of vulnerabilities. The functions (l2TransactionEthCost) and parameters and contracts that are never used or called externally or from inside the contracts should be removed when the contract is deployed on the mainnet.

```
function l2TransactionEthCost(uint256 _l1GasPrice, uint256 _l2GasLimit, uint256
_l2GasPerPubdataByteLimit)
    external
```

```
returns (uint256)

{
    uint256 baseCost = l2TransactionBaseCost(_l1GasPrice, _l2GasLimit,
_l2GasPerPubdataByteLimit);

    return _convertToEthAmount(baseCost);
}
```

Recommendation

If the functions are not supposed to be used anywhere, consider removing them from the contract.

Status

Resolved

Gas

[G-01] Staking#setWithdrawDelay() - Allowing re-storing the same value

Description

The `setWithdrawDelay()` function is found to be allowing re-storing the value in the contract's state variable even when the old value is equal to the new value. This practice results in unnecessary gas consumption due to the `Gsreset` operation (2900 gas), which could be avoided. If the old value and the new value are the same, not updating the storage would avoid this cost and could instead incur a `Gcoldload` (2100 gas) or a `Gwarmaccess` (100 gas), potentially saving gas.

```
function setWithdrawDelay(uint256 newDelay) public onlyRole(MODERATOR_ROLE) {  
    withdrawDelay = newDelay;  
    emit WithdrawDelaySet(newDelay);  
}
```

Recommendation

To optimize gas usage, add a check to compare the old value with the new value before updating the storage. Only perform the storage update if the new value is different from the old value.

Status

Resolved

[G-02] Staking - Gas Optimization in Increments

Description

`++i` costs less gas compared to `i++` or `i += 1` for unsigned integers. In `i++`, the compiler has to create a temporary variable to store the initial value. This is not the case with `++i` in which the value is directly incremented and returned, thus, making it a cheaper alternative.

Recommendation

Consider changing the post-increments (`i++`) to pre-increments (`++i`) as long as the value is not used in any calculations or inside returns.

Status

Acknowledged

[G-03] Staking - Cheaper Conditional Operators**Description**

During compilation, `x != 0` is cheaper than `x > 0` for unsigned integers in solidity inside conditional statements.

Recommendation

Consider using `x != 0` in place of `x > 0` in uint wherever possible.

Status

Acknowledged

[G-04] Staking - Public Constants Can Be Private**Description**

Public constant variables (`MODERATOR_ROLE`) cost more gas because the EVM automatically creates getter functions for them and adds entries to the method ID table. The values can be read from the source code instead.

Recommendation

If reading the values for the constants are not necessary, consider changing the public visibility to private.

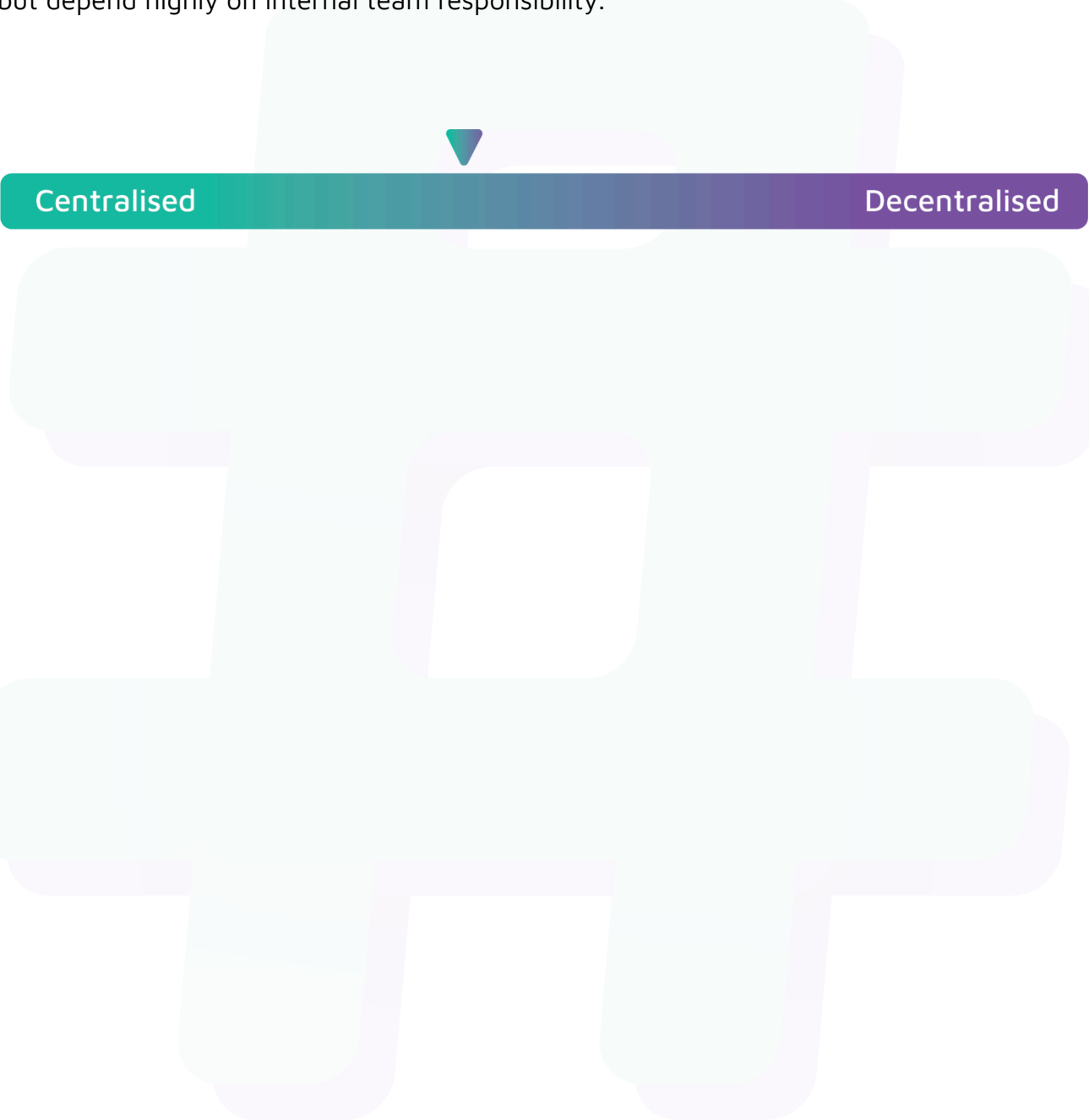
Status

Acknowledged

Centralisation

The OpenZK project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the OpenZK project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.